MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: **RACHELLE MAE M. ORTIZ**

Program: **DOCTOR IN INFORMATION TECHNOLOGY (DIT)**

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in Loading_Dataset_and_CNN.ipynb for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	*2D CNN Model — Dogs vs. Cats Image Classification*
Model Architecture	*2D Convolutional Neural Network (Conv2D)*
Dataset Used	*Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)/ PetImages.zip*
Notebook / File Reference	*Loading_Dataset_and_CNN_Hyperparameter_Tuning.ipynb*
Prepared By	Rachelle Mae M. Ortiz
Date	October 8, 2026
Experiment / Trial No.	Trial 1 (original entry); Trials 2-5 (manual, 50 epochs, no early stopping); Trial 6 (Keras Tuner best)

Trial 1

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	*e.g. 50 / 80 / 100 / 128 / 150*	*100*	*100*	Kept at the notebook value: 100x100 keeps enough detail to tell cats from dogs while keeping 50 epochs affordable on CPU.
Color Mode	*Grayscale / RGB*	*Grayscale*	*Grayscale*	Grayscale is the notebook pipeline (1 channel = smaller, faster model); colour could help but was not changed so trials stay comparable.
Pixel Normalization	*X/255.0 / none*	*X/255.0*	*X/255.0*	Scales pixels to 0-1 so Adam trains faster and more stably.
Shuffle Training Data	*Yes / No*	*Yes (random.shuffle)*	*Yes (random.shuffle)*	Shuffled once with a fixed seed (42) so the stratified split and every trial are reproducible.

Held-out Test Set (separate from training)	*Yes / No*	*No (validation_split only)*	*No (validation_split only)*	A 10 % stratified test set is held out and never used for tuning, so Test Accuracy in section 5 is an unbiased estimate.
Validation Split (of training set)	*0.0 - 0.3*	*0.1*	*0.2*	0.2 of the remaining training data is used for validation (stratified), so over-fitting is visible in val accuracy/loss.

3. Model Architecture Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	*16 / 32 / 64 / 128*	*64*	64	Notebook-style value: 64 filters in the first layer.	Default 0.83 Chosen: 0.49	0.81 0.50
Conv2D Layer 1 – Kernel Size	*(3,3) / (5,5)*	*(3,3)*	*(3,3)*	3x3 is the standard small kernel.		
Conv2D Layer 1 – Activation	*relu / tanh / elu / leaky_relu*	*relu*	*relu*	ReLU.		
Pooling Layer 1 – Type & Pool Size	*MaxPooling2D / AveragePooling 2D; (2,2) / (3,3)*	*MaxPooling 2D, (2,2)*	*MaxPooling2D, (2,2)*	Max pooling 2x2.		
Dense (Fully Connected) Units	*16 / 32 / 64 / 128*	*64*	64	64 units.		
Dropout Rate	*0.0 - 0.5*	*None (not used)*	*None (not used)*	Not used.		
Output Layer Activation	*sigmoid (binary) / softmax (multi-class)*	*sigmoid*	*sigmoid*	sigmoid for binary output.		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	*binary_crossentropy / categorical_crossentropy*	*binary_crossentropy*	*binary_crossentropy*	Binary labels with sigmoid output.	Default 0.83 Chosen: 0.49	0.81 0.50
Optimizer	*adam / sgd / rmsprop*	*adam*	*adam*	Adam (adaptive).		

Learning Rate	*0.1 / 0.01 / 0.001 / 0.0001*	*0.001 (Adam default)*	*0.001 (Adam default)*	Adam default 0.001.		
Batch Size	*8 / 16 / 32 / 64 / 128*	*32*	128	128 to speed up each epoch.		
Number of Epochs	*e.g. 10 - 200*	*200*	10	10 epochs for a quick first run.		
Early Stopping	*Yes (patience = ___) / No*	*No*	No	No early stopping.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy		
Validation Accuracy		
Test Accuracy		
Test Loss		
Precision / Recall / F1 (macro avg)		

5. a. Paste here all screenshots/figures of the tuned model

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Trial 2 - Baseline (original model, 50 epochs)

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	*e.g. 50 / 80 / 100 / 128 / 150*	*100*	*100*	Kept at the notebook value: 100x100 keeps enough detail to tell cats from dogs while keeping 50 epochs affordable on CPU.
Color Mode	*Grayscale / RGB*	*Grayscale*	*Grayscale*	Grayscale is the notebook pipeline (1 channel = smaller, faster model); colour could help but was not changed so trials stay comparable.
Pixel Normalization	*X/255.0 / none*	*X/255.0*	*X/255.0*	Scales pixels to 0-1 so Adam trains faster and more stably.
Shuffle Training Data	*Yes / No*	*Yes (random.shuffle)*	*Yes (random.shuffle, seed 42)*	Shuffled once with a fixed seed (42) so the stratified split and every trial are reproducible.
Held-out Test Set (separate from training)	*Yes / No*	*No (validation_split only)*	*Yes (10 % stratified)*	A 10 % stratified test set is held out and never used for tuning, so Test Accuracy in section 5 is an unbiased estimate.
Validation Split (of training set)	*0.0 - 0.3*	*0.1*	*0.2*	0.2 of the remaining training data is used for validation (stratified), so over-fitting is visible in val accuracy/loss.

3. Model Architecture Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	*16 / 32 / 64 / 128*	*64*	2 (stack 2 - 64 - 64)	Original notebook model: first layer is only 2 filters, so almost nothing is learned early; the 64-filter layers behind it memorise instead.	Default (notebook model): 0.9720 Chosen: 0.9720	Default (notebook model): 0.7566 Chosen: 0.7566
Conv2D Layer 1 – Kernel Size	*(3,3) / (5,5)*	*(3,3)*	*(3,3)*	3x3 kept (standard, cheap).		

Conv2D Layer 1 – Activation	*relu / tanh / elu / leaky_relu*	*relu*	*relu*	ReLU on all conv layers.		
Pooling Layer 1 – Type & Pool Size	*MaxPooling2D / AveragePooling 2D; (2,2) / (3,3)*	*MaxPooling 2D, (2,2)*	*MaxPooli ng2D, (2,2)*	2x2 max pooling after every conv block halves the feature map.		
Dense (Fully Connected) Units	*16 / 32 / 64 / 128*	*64*	64 + 64 (linear, no ReLU)	Two 64-unit Dense layers exactly as in the notebook (they have no activation, so they act as one linear layer).		
Dropout Rate	*0.0 - 0.5*	*None (not used)*	*None (not used)*	Not used, as in the notebook, to demonstrate the over-fit the professor wants to see.		
Output Layer Activation	*sigmoid (binary) / softmax (multi-class)*	*sigmoid*	*sigmoid*	sigmoid: one probability for the binary dog/cat task.		

4. Training Hyperparameters

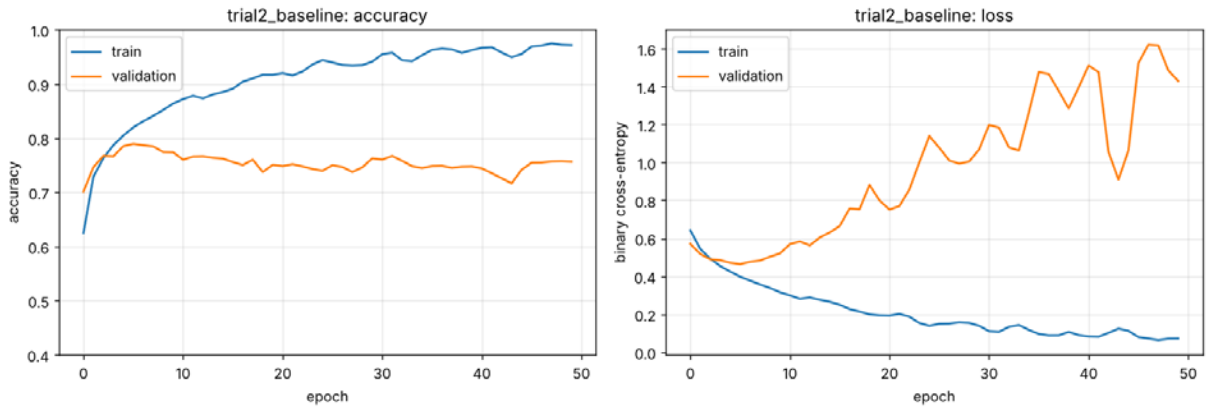
Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	*binary_crossentropy / categorical_crossentropy*	*binary_crossentropy*	*binary_crossentropy*	Binary labels (0/1) with one sigmoid output.	Default (notebook model): 0.9720 Chosen: 0.9720	Default (notebook model): 0.7566 Chosen: 0.7566
Optimizer	*adam / sgd / rmsprop*	*adam*	*adam*	Adam: adaptive, a safe default.		
Learning Rate	*0.1 / 0.01 / 0.001 / 0.0001*	*0.001 (Adam default)*	*0.001 (Adam default)*	Adam default 0.001.		
Batch Size	*8 / 16 / 32 / 64 / 128*	*32*	128	128 as in the notebook run; keeps an epoch ~30-40 s on CPU.		
Number of Epochs	*e.g. 10 - 200*	*200*	50	50 epochs as instructed.		

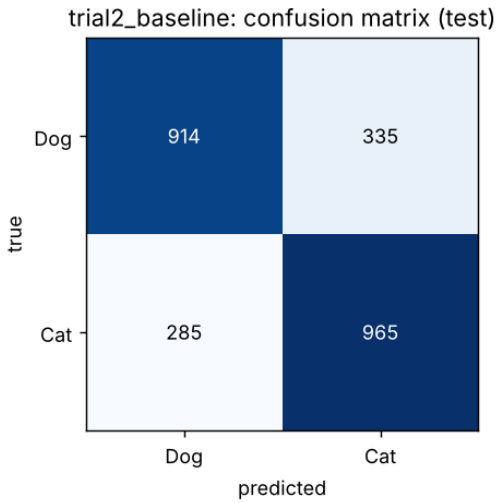
Early Stopping	*Yes (patience = ___) / No*	*No*	No	No early stopping, as instructed, so the full over-fit curve is recorded.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	0.9720	Close to validation (gap <= ~5 pts)
Validation Accuracy	0.7566	>= 0.90 (target)
Test Accuracy	0.7519	>= 0.90, close to validation
Test Loss	1.4635	As low as possible; near the validation loss
Precision / Recall / F1 (macro avg)	0.752 / 0.752 / 0.752	>= 0.90 each

5. a. Paste here all screenshots/figures of the tuned model





6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Result: train 97.2% vs validation 75.7% (gap +21.5 pts); test 75.2%. This is the over-fit model the professor expected.

Evidence: validation loss was lowest at epoch 6 (0.464) and then rose to 1.43 while training loss kept falling.

Best validation accuracy was 79.0% at epoch 6; after that the model only memorised the training images.

Diagnosis: the 2-filter first layer is a bottleneck and the model has no regularisation, so the later layers over-fit.

Next step (Trial 3): widen the first ReLU layers to 16-32-64 and use a ReLU Dense layer.

Early stopping was deliberately not used.

Trial 3 - Wider ReLU conv layers (professor hint)

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	*e.g. 50 / 80 / 100 / 128 / 150*	*100*	*100*	Kept at the notebook value: 100x100 keeps enough detail to tell cats from dogs while keeping 50 epochs affordable on CPU.
Color Mode	*Grayscale / RGB*	*Grayscale*	*Grayscale*	Grayscale is the notebook pipeline (1 channel = smaller, faster model); colour could help but was not changed so trials stay comparable.
Pixel Normalization	*X/255.0 / none*	*X/255.0*	*X/255.0*	Scales pixels to 0-1 so Adam trains faster and more stably.
Shuffle Training Data	*Yes / No*	*Yes (random.shuffle)*	*Yes (random.shuffle, seed 42)*	Shuffled once with a fixed seed (42) so the stratified split and every trial are reproducible.
Held-out Test Set (separate from training)	*Yes / No*	*No (validation_split only)*	*Yes (10 % stratified)*	A 10 % stratified test set is held out and never used for tuning, so Test Accuracy in section 5 is an unbiased estimate.
Validation Split (of training set)	*0.0 - 0.3*	*0.1*	*0.2*	0.2 of the remaining training data is used for validation (stratified), so over-fitting is visible in val accuracy/loss.

3. Model Architecture Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	*16 / 32 / 64 / 128*	*64*	16 (stack 16 - 32 - 64)	Hint from the professor: start near 4 filters and raise the first ReLU layer up to 16. 16-32-64 doubles the filters at each block.	Default (Trial 2 baseline): 0.9720 Chosen: 0.9957	Default (Trial 2 baseline): 0.7566 Chosen: 0.8032
Conv2D Layer 1 – Kernel Size	*(3,3) / (5,5)*	*(3,3)*	*(3,3)*	3x3 kept.		

Conv2D Layer 1 – Activation	*relu / tanh / elu / leaky_relu*	*relu*	*relu*	ReLU everywhere, including the Dense layer (Trial 2 had none).		
Pooling Layer 1 – Type & Pool Size	*MaxPooling2D / AveragePooling 2D; (2,2) / (3,3)*	*MaxPooling 2D, (2,2)*	*MaxPooling2D, (2,2)*	Max pooling 2x2 kept.		
Dense (Fully Connected) Units	*16 / 32 / 64 / 128*	*64*	64 (ReLU)	Single ReLU Dense(64) layer: adds non-linearity with fewer parameters than the two linear layers of Trial 2.		
Dropout Rate	*0.0 - 0.5*	*None (not used)*	*None (not used)*	Still none: this trial isolates the effect of capacity alone.		
Output Layer Activation	*sigmoid (binary) / softmax (multi-class)*	*sigmoid*	*sigmoid*	sigmoid.		

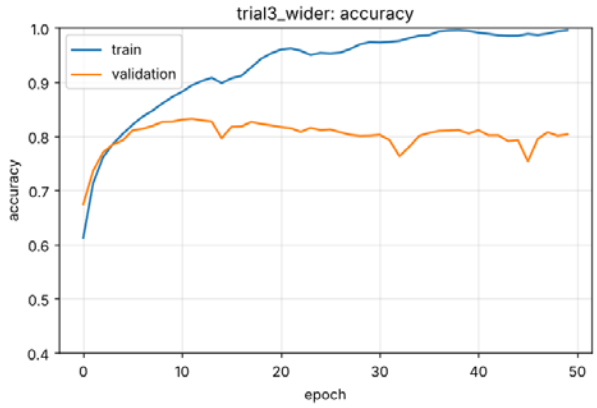
4. Training Hyperparameters

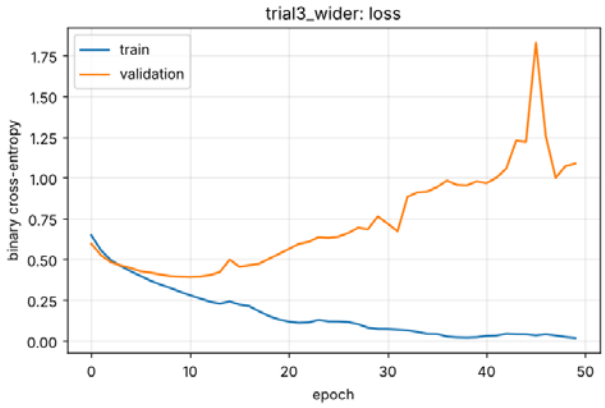
Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	*binary_crossentropy / categorical_crossentropy*	*binary_crossentropy*	*binary_crossentropy*	Unchanged.	Default (Trial 2 baseline): 0.9720 Chosen: 0.9957	Default (Trial 2 baseline): 0.7566 Chosen: 0.8032
Optimizer	*adam / sgd / rmsprop*	*adam*	*adam*	Unchanged.		
Learning Rate	*0.1 / 0.01 / 0.001 / 0.0001*	*0.001 (Adam default)*	*0.001 (Adam default)*	Unchanged.		
Batch Size	*8 / 16 / 32 / 64 / 128*	*32*	128	Unchanged.		
Number of Epochs	*e.g. 10 - 200*	*200*	50	Unchanged (50).		
Early Stopping	*Yes (patience = ___) / No*	*No*	No	No (as instructed).		

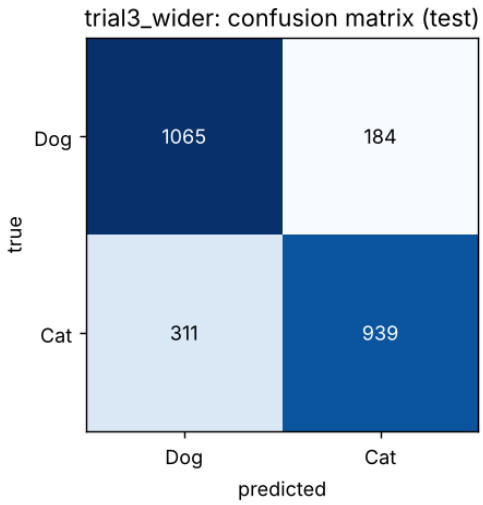
5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	0.9957	Close to validation (gap <= ~5 pts)
Validation Accuracy	0.8032	>= 0.90 (target)
Test Accuracy	0.8019	>= 0.90, close to validation
Test Loss	1.1364	As low as possible; near the validation loss
Precision / Recall / F1 (macro avg)	0.805 / 0.802 / 0.801	>= 0.90 each

5. a. Paste here all screenshots/figures of the tuned model







6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Result: train 99.6%, validation 80.3%, test 80.2%; gap +19.2 pts.

Compared with Trial 2, validation accuracy rose by 4.7 pts, so more filters helped the model learn useful features.

Still severely over-fit: train accuracy reached 99.6% while validation loss climbed from 0.39 (epoch 11) to 1.09.

Capacity alone does not fix over-fitting; regularisation is needed.

Next step (Trial 4): Dropout(0.5).

Trial 4 - Add Dropout(0.5)

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	*e.g. 50 / 80 / 100 / 128 / 150*	*100*	*100*	Kept at the notebook value: 100x100 keeps enough detail to tell cats from dogs while keeping 50 epochs affordable on CPU.
Color Mode	*Grayscale / RGB*	*Grayscale*	*Grayscale*	Grayscale is the notebook pipeline (1 channel = smaller, faster model); colour could help but was not changed so trials stay comparable.
Pixel Normalization	*X/255.0 / none*	*X/255.0*	*X/255.0*	Scales pixels to 0-1 so Adam trains faster and more stably.
Shuffle Training Data	*Yes / No*	*Yes (random.shuffle)*	*Yes (random.shuffle, seed 42)*	Shuffled once with a fixed seed (42) so the stratified split and every trial are reproducible.
Held-out Test Set (separate from training)	*Yes / No*	*No (validation_split only)*	*Yes (10 % stratified)*	A 10 % stratified test set is held out and never used for tuning, so Test Accuracy in section 5 is an unbiased estimate.
Validation Split (of training set)	*0.0 - 0.3*	*0.1*	*0.2*	0.2 of the remaining training data is used for validation (stratified), so over-fitting is visible in val accuracy/loss.

3. Model Architecture Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	*16 / 32 / 64 / 128*	*64*	16 (stack 16 - 32 - 64)	Same conv stack as Trial 3 so the change is attributable to regularisation.	Default (Trial 2 baseline): 0.9720 Chosen: 0.9889	Default (Trial 2 baseline): 0.7566 Chosen: 0.8230
Conv2D Layer 1 – Kernel Size	*(3,3) / (5,5)*	*(3,3)*	*(3,3)*	3x3.		
Conv2D Layer 1 – Activation	*relu / tanh / elu / leaky_relu*	*relu*	*relu*	ReLU.		

Pooling Layer 1 – Type & Pool Size	*MaxPooling2D / AveragePooling 2D; (2,2) / (3,3)*	*MaxPooling 2D, (2,2)*	*MaxPooli ng2D, (2,2)*	Max pooling 2x2.		
Dense (Fully Connected) Units	*16 / 32 / 64 / 128*	*64*	128 (ReLU)	Dense raised to 128 because Dropout removes half of the units on each step.		
Dropout Rate	*0.0 - 0.5*	*None (not used)*	*0.5 (after Dense)*	Dropout 0.5 on the Dense layer to reduce co-adaptation and shrink the train/validation gap.		
Output Layer Activation	*sigmoid (binary) / softmax (multi-class)*	*sigmoid*	*sigmoid*	sigmoid.		

4. Training Hyperparameters

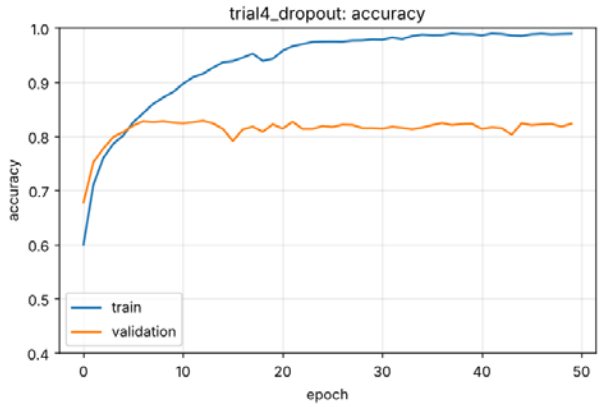
Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	*binary_crossentropy / categorical_crossentropy*	*binary_crossentropy*	*binary_crossentropy*	Unchanged.	Default (Trial 2 baseline): 0.9720 Chosen: 0.9889	Default (Trial 2 baseline): 0.7566 Chosen: 0.8230
Optimizer	*adam / sgd / rmsprop*	*adam*	*adam*	Unchanged.		
Learning Rate	*0.1 / 0.01 / 0.001 / 0.0001*	*0.001 (Adam default)*	*0.001 (Adam default)*	Unchanged.		
Batch Size	*8 / 16 / 32 / 64 / 128*	*32*	128	Unchanged.		
Number of Epochs	*e.g. 10 - 200*	*200*	50	Unchanged (50).		
Early Stopping	*Yes (patience = ___) / No*	*No*	No	No (as instructed).		

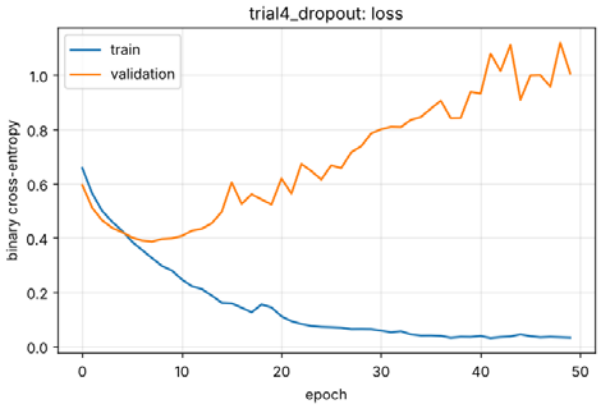
5. Evaluation Results (after training with chosen values) - Final Model

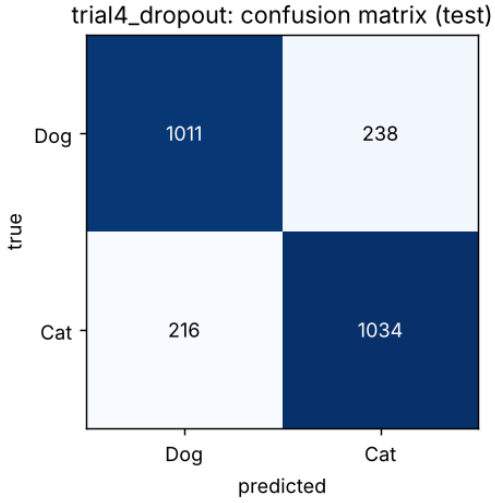
Metric	Value Obtained	Target / Acceptable Range

Training Accuracy	0.9889	Close to validation (gap <= ~5 pts)
Validation Accuracy	0.8230	>= 0.90 (target)
Test Accuracy	0.8183	>= 0.90, close to validation
Test Loss	0.9973	As low as possible; near the validation loss
Precision / Recall / F1 (macro avg)	0.818 / 0.818 / 0.818	>= 0.90 each

5. a. Paste here all screenshots/figures of the tuned model







6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Result: train 98.9%, validation 82.3%, test 81.8%; gap +16.6 pts.

Dropout gave only a small gain over Trial 3 (+2.0 pts validation) and the gap stayed large.

Validation loss still rose after epoch 8 (0.39 -> 1.01): the network sees only ~18,000 distinct images and memorises them.

Lesson: dropout alone is not enough here; the model needs more varied training data.

Next step (Trial 5): add a 4th conv layer and data augmentation.

Trial 5 - Extra conv layer + data augmentation

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	*e.g. 50 / 80 / 100 / 128 / 150*	*100*	*100*	Kept at the notebook value: 100x100 keeps enough detail to tell cats from dogs while keeping 50 epochs affordable on CPU.
Color Mode	*Grayscale / RGB*	*Grayscale*	*Grayscale*	Grayscale is the notebook pipeline (1 channel = smaller, faster model); colour could help but was not changed so trials stay comparable.
Pixel Normalization	*X/255.0 / none*	*X/255.0*	*X/255.0*	Scales pixels to 0-1 so Adam trains faster and more stably.
Shuffle Training Data	*Yes / No*	*Yes (random.shuffle)*	*Yes (random.shuffle, seed 42)*	Shuffled once with a fixed seed (42) so the stratified split and every trial are reproducible.
Held-out Test Set (separate from training)	*Yes / No*	*No (validation_split only)*	*Yes (10 % stratified)*	A 10 % stratified test set is held out and never used for tuning, so Test Accuracy in section 5 is an unbiased estimate.
Validation Split (of training set)	*0.0 - 0.3*	*0.1*	*0.2*	0.2 of the remaining training data is used for validation (stratified), so over-fitting is visible in val accuracy/loss.

3. Model Architecture Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	*16 / 32 / 64 / 128*	*64*	16 (stack 16 - 32 - 64 - 128)	Fourth conv block (128 filters) added ("add another layer until we can tune"); first layer stays at 16.	Default (Trial 2 baseline): 0.9720 Chosen: 0.9116	Default (Trial 2 baseline): 0.7566 Chosen: 0.8955
Conv2D Layer 1 – Kernel Size	*(3,3) / (5,5)*	*(3,3)*	*(3,3)*	3x3.		
Conv2D Layer 1 – Activation	*relu / tanh / elu / leaky_relu*	*relu*	*relu*	ReLU.		

Pooling Layer 1 – Type & Pool Size	*MaxPooling2D / AveragePooling 2D; (2,2) / (3,3)*	*MaxPooling 2D, (2,2)*	*MaxPooli ng2D, (2,2)*	Max pooling 2x2 after each of the 4 blocks (feature map 100 -> 4).		
Dense (Fully Connected) Units	*16 / 32 / 64 / 128*	*64*	128 (ReLU)	Dense(128) as in Trial 4.		
Dropout Rate	*0.0 - 0.5*	*None (not used)*	*0.5 (after Dense)*	Dropout 0.5 kept, now combined with data augmentation (random horizontal flip, +/-8 % rotation, 10 % zoom; training only).		
Output Layer Activation	*sigmoid (binary) / softmax (multi-class)*	*sigmoid*	*sigmoid*	sigmoid.		

4. Training Hyperparameters

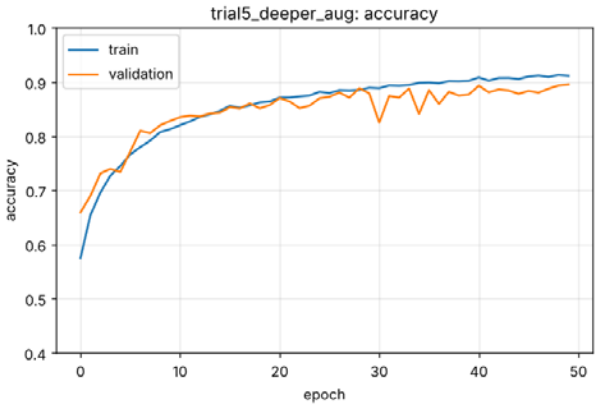
Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	*binary_crossentropy / categorical_crossentropy*	*binary_crossentropy*	*binary_crossentropy*	Unchanged.	Default (Trial 2 baseline): 0.9720 Chosen: 0.9116	Default (Trial 2 baseline): 0.7566 Chosen: 0.8955
Optimizer	*adam / sgd / rmsprop*	*adam*	*adam*	Unchanged.		
Learning Rate	*0.1 / 0.01 / 0.001 / 0.0001*	*0.001 (Adam default)*	*0.001 (Adam default)*	Unchanged.		
Batch Size	*8 / 16 / 32 / 64 / 128*	*32*	128	Unchanged.		
Number of Epochs	*e.g. 10 - 200*	*200*	50	Unchanged (50).		
Early Stopping	*Yes (patience = ___) / No*	*No*	No	No (as instructed).		

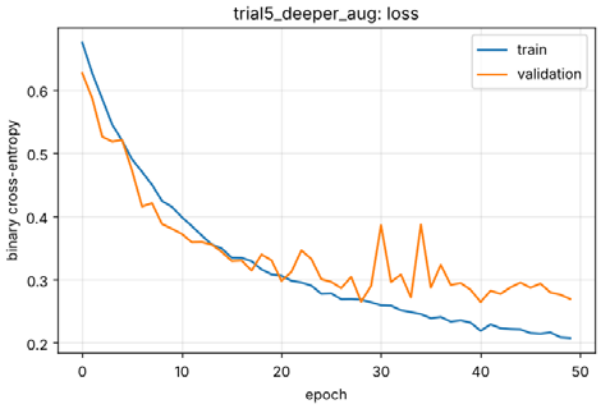
5. Evaluation Results (after training with chosen values) - Final Model

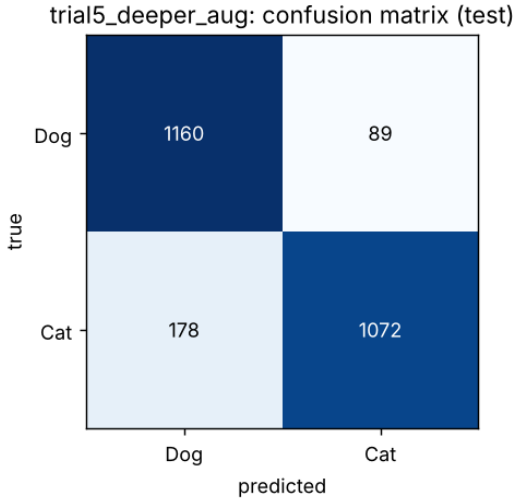
Metric	Value Obtained	Target / Acceptable Range

Training Accuracy	0.9116	Close to validation (gap <= ~5 pts)
Validation Accuracy	0.8955	>= 0.90 (target)
Test Accuracy	0.8932	>= 0.90, close to validation
Test Loss	0.2682	As low as possible; near the validation loss
Precision / Recall / F1 (macro avg)	0.895 / 0.893 / 0.893	>= 0.90 each

5. a. Paste here all screenshots/figures of the tuned model







6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Result: train 91.2%, validation 89.6%, test 89.3%; gap only +1.6 pts (Trial 2: +21.5). This is the best-fit manual model.

Test loss is 0.268 versus 1.46 for the baseline, and macro F1 is 0.893.

Augmentation was the decisive change: both curves improve together and validation loss no longer climbs.

The 90 % target was not fully reached (89.6% validation / 89.3% test). Accuracy was still slowly improving at epoch 50 (best val acc at epoch 50), so more epochs, a learning-rate schedule or colour input are the next things to try.

No early stopping was used, as instructed; the reported numbers are the final-epoch model.

Trade-off: ~48 min of CPU time for 50 epochs (augmentation makes each epoch slower).

Trial 6 - Keras Tuner (RandomSearch) best configuration

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	*e.g. 50 / 80 / 100 / 128 / 150*	*100*	*100*	Kept at the notebook value: 100x100 keeps enough detail to tell cats from dogs while keeping 50 epochs affordable on CPU.
Color Mode	*Grayscale / RGB*	*Grayscale*	*Grayscale*	Grayscale is the notebook pipeline (1 channel = smaller, faster model); colour could help but was not changed so trials stay comparable.
Pixel Normalization	*X/255.0 / none*	*X/255.0*	*X/255.0*	Scales pixels to 0-1 so Adam trains faster and more stably.
Shuffle Training Data	*Yes / No*	*Yes (random.shuffle)*	*Yes (random.shuffle, seed 42)*	Shuffled once with a fixed seed (42) so the stratified split and every trial are reproducible.
Held-out Test Set (separate from training)	*Yes / No*	*No (validation_split only)*	*Yes (10 % stratified)*	A 10 % stratified test set is held out and never used for tuning, so Test Accuracy in section 5 is an unbiased estimate.
Validation Split (of training set)	*0.0 - 0.3*	*0.1*	*0.2*	0.2 of the remaining training data is used for validation (stratified), so over-fitting is visible in val accuracy/loss.

3. Model Architecture Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	*16 / 32 / 64 / 128*	*64*	4 (stack 4 - 8 - 16 - 32)	Chosen by Keras Tuner. Search space: 2-4 conv blocks, base filters 4/8/16/32 (doubling per block), RandomSearch, 8 candidates x 10 epochs, objective = val_accuracy.	Default (Trial 2 baseline): 0.9720 Chosen: 0.9673	Default (Trial 2 baseline): 0.7566 Chosen: 0.7599
Conv2D Layer 1 – Kernel Size	*(3,3) / (5,5)*	*(3,3)*	*(3,3)*	3x3 (fixed in the search).		

Conv2D Layer 1 – Activation	*relu / tanh / elu / leaky_relu*	*relu*	*relu*	ReLU (fixed in the search).		
Pooling Layer 1 – Type & Pool Size	*MaxPooling2D / AveragePooling 2D; (2,2) / (3,3)*	*MaxPooling 2D, (2,2)*	*MaxPooli ng2D, (2,2)*	Max pooling 2x2 (fixed in the search).		
Dense (Fully Connected) Units	*16 / 32 / 64 / 128*	*64*	128 (ReLU)	Tuner searched 32 / 64 / 128 and picked 128.		
Dropout Rate	*0.0 - 0.5*	*None (not used)*	*None (0.0)*	Tuner searched 0.0-0.5 and picked 0.0.		
Output Layer Activation	*sigmoid (binary) / softmax (multi-class)*	*sigmoid*	*sigmoid*	sigmoid.		

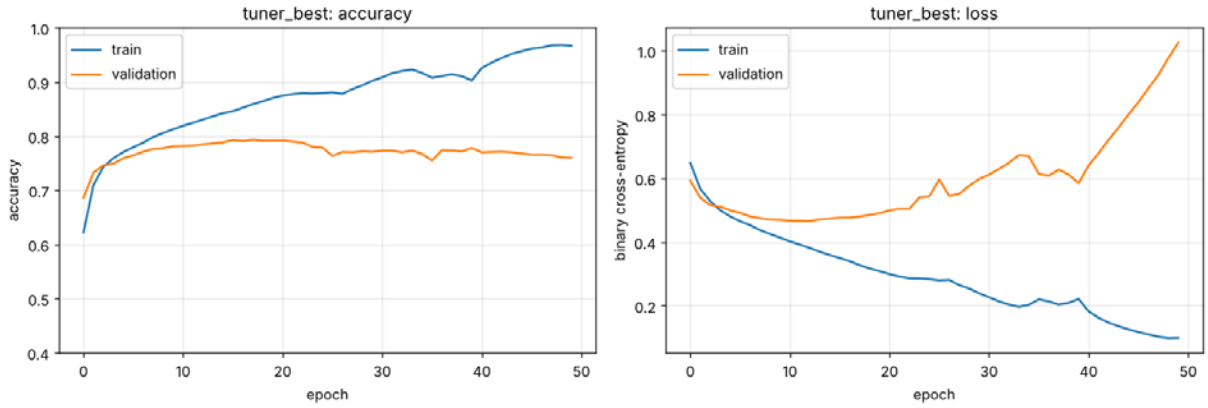
4. Training Hyperparameters

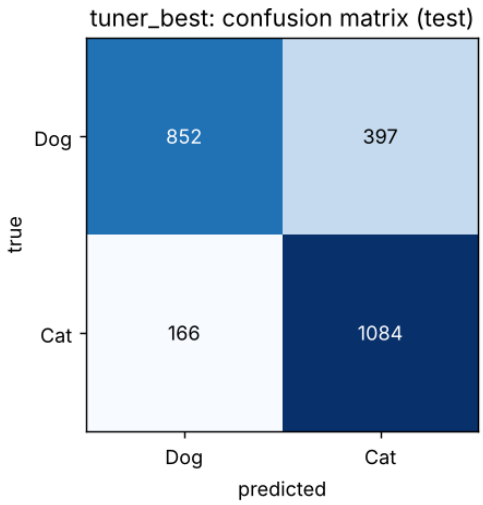
Hyperparameter	**Options / Range Considered**	**Notebook Default**	**Chosen Value**	**Justification**	**Training Accuracy**	**Validation Accuracy**
Loss Function	*binary_crossentropy / categorical_crossentropy*	*binary_crossentropy*	*binary_crossentropy*	Unchanged.		
Optimizer	*adam / sgd / rmsprop*	*adam*	*adam*	Unchanged.		
Learning Rate	*0.1 / 0.01 / 0.001 / 0.0001*	*0.001 (Adam default)*	*0.001 (Adam default)*	Tuner searched 0.01 / 0.001 / 0.0001 and picked 0.001.		
Batch Size	*8 / 16 / 32 / 64 / 128*	*32*	128	128 (fixed).	Default (Trial 2 baseline): 0.9720 Chosen: 0.9673	Default (Trial 2 baseline): 0.7566 Chosen: 0.7599
Number of Epochs	*e.g. 10 - 200*	*200*	50	Search used 10 epochs per candidate; the winner was retrained for 50 epochs.		
Early Stopping	*Yes (patience = ___) / No*	*No*	No	No (as instructed).		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	0.9673	Close to validation (gap <= ~5 pts)
Validation Accuracy	0.7599	>= 0.90 (target)
Test Accuracy	0.7747	>= 0.90, close to validation
Test Loss	0.9695	As low as possible; near the validation loss
Precision / Recall / F1 (macro avg)	0.784 / 0.775 / 0.773	>= 0.90 each

5. a. Paste here all screenshots/figures of the tuned model





6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Result: train 96.7%, validation 76.0%, test 77.5%; gap +20.7 pts.

Keras Tuner chose a small, un-regularised network: 4 conv blocks (4-8-16-32), Dense(128), no dropout, no augmentation, lr 0.001.

Likely reason: every candidate was trained for only 10 epochs. Over a short budget the tuner favours models that learn fast, which probably penalised augmentation (it learns more slowly but generalised better over 50 epochs in Trial 5).

Over 50 epochs this model over-fits again (validation loss min 0.47 at epoch 13 -> 1.03).

The manual Trial 5 is better by 13.6 pts validation and 11.8 pts test.

A longer search budget (more epochs per candidate, e.g. Hyperband) or a search space that forces augmentation/dropout would likely close the gap.

7. Manual vs. Keras Tuner Comparison

Run	Configuration	Params	Train acc	Val acc	Gap (pts)	Test acc	Test loss	Macro F1	Time (min)
Trial 2	Baseline (2-64-64), no dropout	452,053	0.9720	0.7566	+21.5	0.7519	1.4635	0.752	32
Trial 3	16-32-64, ReLU dense	433,025	0.9957	0.8032	+19.2	0.8019	1.1364	0.801	29
Trial 4	16-32-64 + Dropout 0.5	842,753	0.9889	0.8230	+16.6	0.8183	0.9973	0.818	29
Trial 5	16-32-64-128 + Dropout 0.5 + augmentation	359,553	0.9116	0.8955	+1.6	0.8932	0.2682	0.893	49
Trial 6 (Keras Tuner)	4-8-16-32, Dense 128, no dropout	71,937	0.9673	0.7599	+20.7	0.7747	0.9695	0.773	13

Effort: manual tuning needed 4 trials x 50 epochs = 139 min of training plus the human decisions between trials. Keras Tuner searched 8 random candidates x 10 epochs in 24 min with no human input, then 13 min to retrain the winner for 50 epochs.
Quality: the best manual model (Trial 5) reached 89.6% validation / 89.3% test with a 1.6-pt gap. The Keras Tuner model reached 76.0% validation / 77.5% test with a 20.7-pt gap, i.e. it over-fits again.
Why the difference: manual tuning used knowledge that the search space and budget did not encode - over-fitting is the problem, so add regularisation and augmentation, and judge models over the full 50 epochs. The tuner judged candidates after only 10 epochs, so it likely favoured fast-learning, un-regularised networks (the augmented candidates ranked lower, see the table below). This explanation is a hypothesis; it was not tested with a longer search.
Takeaway: Keras Tuner is fast and systematic for exploring many combinations, but its answer is only as good as its search space, its training budget and its objective. Manual reasoning from the learning curves found the better fit here; the two approaches are best combined (manual reasoning to design the space, the tuner to refine within it).

Keras Tuner search results (8 candidates x 10 epochs, ranked by best val_accuracy)

Conv blocks	Base filters	Dense units	Dropout	Learning rate	Augmentation	Val acc (10 ep)
4	4	128	0.0	0.001	no	0.7841
2	8	128	0.2	0.001	no	0.7823
2	8	128	0.2	0.001	yes	0.7470
2	4	64	0.4	0.001	yes	0.7354
2	4	128	0.3	0.0001	no	0.7268
3	4	64	0.1	0.0001	no	0.7021
3	4	64	0.1	0.0001	yes	0.6494
4	4	32	0.2	0.01	no	0.5002

